

AI Fraud Detection for UPI Transactions

Real-Time Anomaly Detection + Multi-Model Ensemble + GenAI Fraud Explainer + Live Alert System

1. PROJECT OVERVIEW

1.1 Background & Motivation

India processes over 10 billion UPI transactions every month — making it the world's largest real-time payments network. With this scale comes an equally massive fraud surface. Scammers exploit UPI through phishing VPAs (Virtual Payment Addresses), SIM swap attacks, social engineering, mule account networks, and transaction velocity manipulation. The National Payments Corporation of India (NPCI) and banks are in a continuous arms race against increasingly sophisticated fraud techniques.

Traditional rule-based fraud detection systems — 'block if amount > ₹50,000 from a new device' — are brittle, easy to evade, and generate enormous false-positive rates that frustrate legitimate users. What is needed is a system that learns the normal behaviour of every UPI participant and detects statistical deviations in real time, before the transaction completes.

This project challenges you to build a production-grade AI Fraud Detection Platform for UPI transactions. You will collect and self-build UPI transaction data, train a multi-model anomaly detection ensemble, detect suspicious transactions in real time from a streaming Kafka feed, generate risk scores with explainability, and alert a fraud analyst through a live monitoring dashboard — with GenAI-powered fraud explanations streamed per alert.

1.2 Problem Statement

"A UPI transaction arrives on the streaming pipeline. Within 200 milliseconds, the fraud detection engine must assign a risk score, classify it as Legitimate / Suspicious / High-Risk / Block, stream a GenAI fraud explanation to the analyst dashboard, and if High-Risk or Block — trigger an automatic alert via webhook to the payment gateway to hold the transaction pending review."

1.3 High-Level Goals

- Self-build a realistic UPI transaction dataset: collect, simulate, and label at least 500,000 transactions with realistic Indian UPI patterns and fraud scenarios.
- Engineer UPI-specific features: velocity features, device fingerprint consistency, VPA age, merchant category risk, geographic anomaly, time-of-day behaviour, and graph-based network features.

- ❑ **Train a multi-model anomaly detection ensemble: Isolation Forest + Autoencoder + XGBoost classifier — each contributing to a final ensemble risk score.**
- ❑ **Handle extreme class imbalance: fraud is typically 0.1–0.5% of transactions. The system must achieve high recall on fraud without generating unacceptable false positive rates.**
- ❑ **Detect fraud in real time from a Kafka stream — inference latency must be < 200ms per transaction end-to-end.**
- ❑ **Classify every transaction into one of four risk tiers: Legitimate, Suspicious, High-Risk, Block.**
- ❑ **Stream a GenAI fraud explanation for every Suspicious, High-Risk, or Block transaction to the analyst dashboard.**
- ❑ **Integrate a simulated Payment Gateway webhook: High-Risk and Block transactions trigger an automatic hold request.**
- ❑ **Build a live Fraud Operations Centre dashboard: real-time transaction feed, alert queue, network graph visualisation, model performance metrics.**
- ❑ **Expose full observability: inference latency, false positive rate, fraud catch rate, model drift detection.**

2. SCOPE & CONSTRAINTS

2.1 In Scope

- ❑ UPI-specific transaction fraud detection — peer-to-peer (P2P) and peer-to-merchant (P2M) transactions.
- ❑ Multi-model ensemble: Isolation Forest (unsupervised anomaly detection) + Deep Autoencoder (reconstruction error) + XGBoost (supervised classifier).
- ❑ Real-time inference via Kafka streaming with < 200ms end-to-end latency target.
- ❑ Four-tier risk classification: Legitimate (green) / Suspicious (amber) / High-Risk (orange) / Block (red).
- ❑ SHAP explainability: every fraud prediction accompanied by the top 5 features that drove the risk score.
- ❑ GenAI fraud narrative: streamed natural language explanation of why a transaction was flagged, what fraud pattern it matches, and what the analyst should verify.
- ❑ Payment Gateway webhook simulator: High-Risk and Block transactions trigger an automatic hold request to a mock gateway API.
- ❑ Graph-based fraud network detection: build a transaction graph and detect suspicious clusters (mule account rings, rapid fund forwarding chains).
- ❑ Model drift monitoring: detect when the model's fraud catch rate drops, indicating the fraud pattern has evolved.
- ❑ Full observability: Prometheus metrics, Grafana dashboards, MLflow model tracking, structured logs.

2.2 Out of Scope

- ❑ Integration with real NPCI infrastructure or live UPI APIs.
- ❑ IMPS, NEFT, or credit card fraud detection — UPI only.
- ❑ Customer-facing fraud dispute resolution workflows.
- ❑ Biometric or device authentication.

- ❑ Multi-language support (English only for the UI and GenAI explanations).
- ❑ Mobile application — web dashboard only.

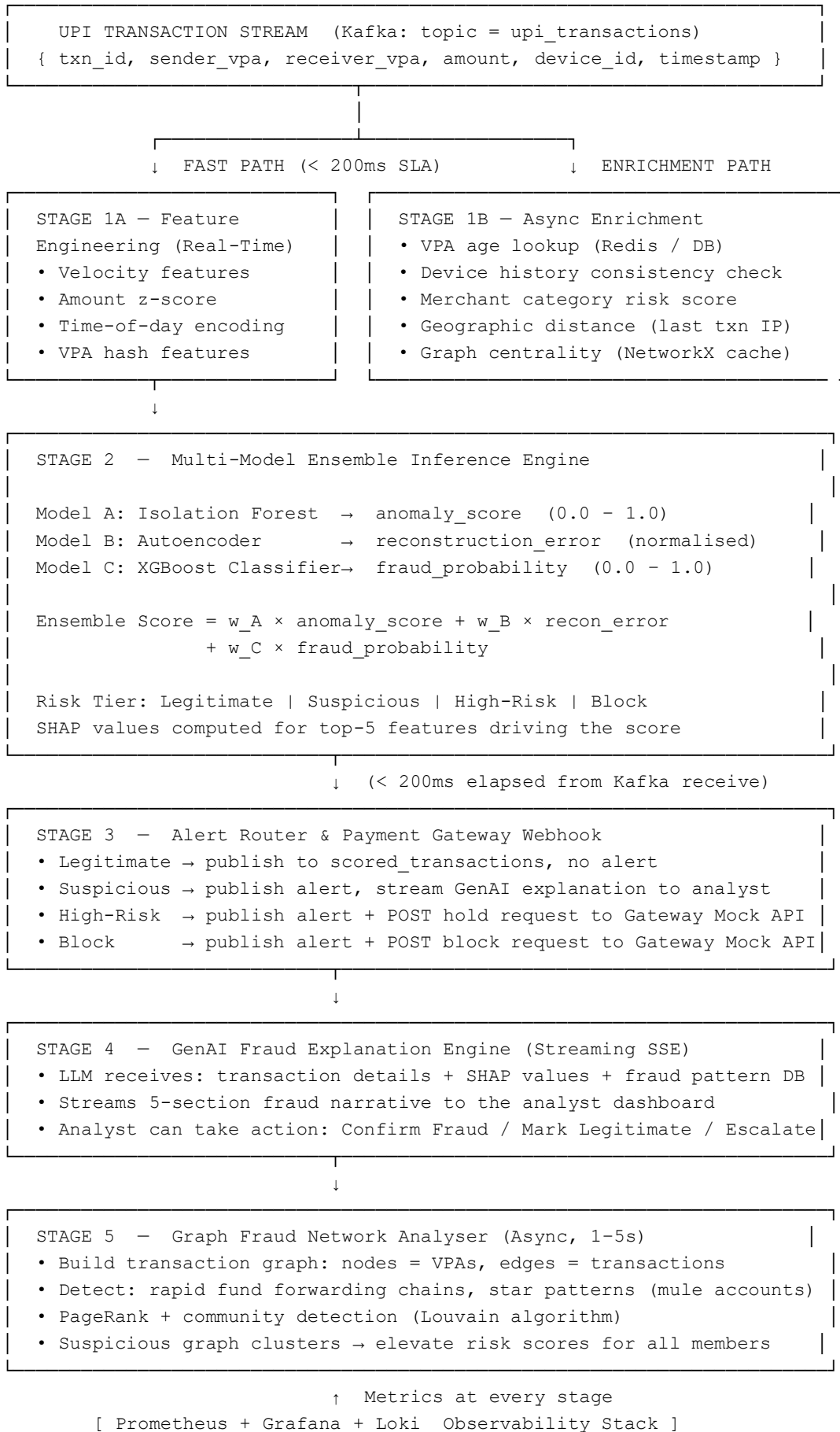
2.3 Hard Constraints

< 200ms Inference	The fraud scoring pipeline from Kafka message received to risk label assigned must complete in under 200 milliseconds. This is a hard production SLA — if the system cannot score a transaction before the payment network times out, it is useless.
Ensemble Required	A single model is not acceptable. You must train all three model types (Isolation Forest, Autoencoder, XGBoost) and combine their scores into an ensemble. Demonstrate each model's individual contribution during the demo.
Class Imbalance Handling	Simply training on imbalanced data and reporting 99.9% accuracy is not acceptable. You must apply at least two imbalance-handling techniques (SMOTE, cost-sensitive learning, threshold tuning) and justify your choices with precision-recall curves.
SHAP Explainability	Every fraud prediction must be accompanied by SHAP values. A black-box score with no explanation is not acceptable for a fraud operations context.
Self-Built Dataset	You must build and label your own dataset. You may use the IEEE-CIS Fraud Detection dataset or PaySim for structural inspiration, but your final training data must be self-generated with UPI-specific features and Indian payment context.
Graph Features Required	Network/graph features must be computed for every transaction. A system with only tabular features misses the most powerful signal in UPI fraud — the network topology of fund flows between VPAs.

3. SYSTEM ARCHITECTURE

3.1 Architecture Overview

The platform is a real-time streaming fraud detection system with three parallel processing paths: the fast inference path (< 200ms), the enrichment and graph path (async, 1–5 seconds), and the GenAI explanation path (streamed to analyst). All paths feed the live Fraud Operations Centre dashboard.



3.2 Component Deep Dives

3.2.1 UPI Transaction Simulator & Stream Producer

Since no real UPI transaction stream is available, you must build a realistic simulator that generates authentic UPI transaction patterns. Requirements:

- Simulate transactions across 10,000 unique VPAs (Virtual Payment Addresses) — structured as realistic Indian UPI IDs (e.g., user@oksbi, merchant@ybl, company@ibl).
- Implement realistic legitimate transaction patterns: salary credits (1st/30th of month), bill payments (recurring, fixed amount), small P2P transfers, merchant payments.
- Implement fraud scenarios — each must be detectable by different model components:
 - (Velocity fraud: 50+ transactions from one VPA within 10 minutes (detectable by velocity features).
 - (Large amount anomaly: a ₹2,00,000 transfer from a VPA that normally sends < ₹500 (detectable by z-score feature).
 - (New device fraud: transaction from a device_id never seen for this VPA (detectable by device consistency).
 - (Mule account ring: rapid fund forwarding A → B → C → D → E within 5 minutes (detectable by graph features).
 - (Night-time anomaly: transaction at 3 AM from a VPA whose entire history is 9AM–9PM (detectable by time features).
 - (VPA impersonation: transaction from a new VPA very similar to a known merchant (e.g., amaz0n@ybl vs. amazon@ybl).
- Configurable fraud injection rate: default 0.3% of transactions are fraudulent, configurable 0.1–5%.
- Publish to Kafka topic: upi_transactions at configurable rate (default: 500 txns/second to stress-test the < 200ms SLA).

3.2.2 Feature Engineering Pipeline

UPI fraud detection requires a rich feature set that goes far beyond the raw transaction fields. The feature engineering service must compute the following feature groups for every transaction:

Feature Group	Features	Computation Source
Velocity Features	txn_count_1min, txn_count_5min, txn_count_1hr, amount_sum_1hr, unique_receivers_1hr, unique_devices_1hr	Redis sliding window counters
Amount Features	amount_zscore_sender, amount_zscore_receiver, amount_vs_daily_avg, amount_vs_monthly_avg, is_round_number	PostgreSQL history + Redis
Temporal Features	hour_of_day, day_of_week, is_weekend, is_night (10PM–6AM), days_since_last_txn, is_first_txn_ever	Timestamp computation
Device Features	is_new_device, device_txn_count, device_vpa_count, device_last_seen_hours_ago, device_risk_score	Device history DB
VPA Features	sender_vpa_age_days, receiver_vpa_age_days, sender_txn_history_size, receiver_txn_history_size, vpa_name_similarity_score	VPA registry DB

Merchant Features	is_merchant_txn, merchant_category_risk, merchant_age_days, merchant_dispute_rate, merchant_avg_txn_amount	Merchant catalog DB
Geographic Features	ip_country, ip_state, distance_from_last_txn_km, is_geo_impossible (speed > 900 km/h between txns)	IP geolocation DB
Graph Features	sender_pagerank, receiver_pagerank, community_id, is_in_suspicious_cluster, chain_length (hops to known fraud VPA)	NetworkX graph (cached Redis)

Features required for the fast path (< 200ms): all Velocity, Amount, Temporal, and Device features — these are precomputed and cached in Redis. Graph and Geographic features are computed asynchronously and used to update the risk score within 5 seconds, potentially escalating a Suspicious to High-Risk.

3.2.3 Multi-Model Ensemble Inference Engine

Three models are trained independently and their outputs combined via a weighted ensemble. Each model detects a different type of anomaly:

Model A — Isolation Forest (Unsupervised Anomaly Detection)

- Trained on legitimate transactions ONLY — no fraud labels needed. Detects transactions that are statistically isolated (hard to reach from the rest of the data).
- Features used: all tabular features (velocity, amount z-scores, temporal, device, VPA age).
- Output: anomaly_score (0.0 = normal, 1.0 = maximally anomalous). Score is normalised using min-max scaling from the training distribution.
- Advantage: catches novel fraud patterns not seen during training. Disadvantage: no concept of 'what type' of fraud.

Model B — Deep Autoencoder (Reconstruction Error)

- A neural network trained to reconstruct legitimate transactions. Fraud transactions — being out-of-distribution — produce high reconstruction error.
- Architecture: Input layer → Dense(64, ReLU) → Dense(32, ReLU) → Dense(16, ReLU) [bottleneck] → Dense(32, ReLU) → Dense(64, ReLU) → Output layer.
- Trained on normalised legitimate transaction vectors. Loss function: Mean Squared Error between input and reconstruction.
- Output: reconstruction_error per transaction. Normalised against the 95th percentile of error from the validation set.
- Advantage: captures complex non-linear patterns across all features simultaneously.

Model C — XGBoost Supervised Classifier

- Trained on labelled data (legitimate vs. fraud). Uses SMOTE oversampling + class_weight balancing to address the extreme class imbalance.
- Features: full feature set including graph features (added after async enrichment for offline retraining).

- Output: fraud_probability (0.0–1.0). Threshold tuned on validation set to optimise F2-score (recall-weighted, since missing fraud is worse than false alarms).
- Output also includes SHAP values for every prediction — the top 5 features with their contribution direction and magnitude.
- Advantage: highest precision when trained data covers the fraud patterns. Disadvantage: struggles with novel fraud types not in training data.

Ensemble Combination

The final risk score is a weighted average of the three model outputs. Weights are tuned on the validation set to maximise the F2-score:

```
ensemble_score = (0.25 × anomaly_score)
                + (0.25 × normalised_reconstruction_error)
                + (0.50 × fraud_probability)
```

Risk Tier Assignment:

```
ensemble_score < 0.30 → Legitimate    (green - no action)
ensemble_score < 0.55 → Suspicious    (amber - analyst alert)
ensemble_score < 0.80 → High-Risk     (orange - gateway hold + alert)
ensemble_score >= 0.80 → Block        (red - gateway block + alert)
```

Note: Thresholds are configurable per transaction type (P2P vs. P2M) and per time-of-day (night transactions use lower thresholds).

3.2.4 Graph Fraud Network Analyser

The graph analyser builds and maintains a directed transaction graph where nodes are VPAs and edges are transactions. It runs asynchronously and can escalate risk scores after the initial fast-path decision.

- Graph Library: NetworkX (Python) — build in-memory graph; persist adjacency list to Redis for fast lookup.
- PageRank: Compute PageRank for all VPA nodes. A high PageRank on the receiver side of a transaction is a strong fraud signal (money flowing to a central aggregator — classic mule account pattern).
- Community Detection: Use the Louvain algorithm to detect tightly connected clusters of VPAs. A transaction that connects two previously separate clusters is flagged.
- Rapid Fund Forwarding Detection: Use BFS/DFS to detect chains where money passes through 3+ VPAs within 10 minutes — a classic money laundering pattern in UPI fraud.
- Star Pattern Detection: A VPA receiving funds from 20+ unique senders within 1 hour — typical of a mule account collecting from phishing victims.
- Known Fraud VPA Distance: Compute the minimum hop count from any transaction participant to a known confirmed-fraud VPA in the database. Distance = 1 (direct) is an immediate Block signal.

3.2.5 GenAI Fraud Explanation Engine

Every Suspicious, High-Risk, or Block transaction triggers the GenAI engine to produce a fraud analyst report streamed token-by-token. The LLM receives:

- Full transaction details (amount, VPAs, device, timestamp, location).
- The ensemble risk score and each model's individual score.
- SHAP values: top 5 features and their contribution direction.
- Graph context: the transaction's position in the fraud network (PageRank, community, chain length).

- ❑ Matched fraud pattern (from a fraud pattern database — e.g., 'VELOCITY_SPIKE', 'MULE_ACCOUNT', 'NEW_DEVICE_LARGE_AMOUNT').

The GenAI engine must produce a 5-section streaming fraud analyst report:

Section	Required Content	Approx. Length
1. Fraud Summary	One-paragraph summary: risk tier, ensemble score, primary reason for flagging, recommended immediate action	1 paragraph
2. Anomaly Breakdown	Explain each model's signal: what specifically did Isolation Forest / Autoencoder / XGBoost detect? What features drove the score?	3 sub-sections
3. Transaction Pattern	What fraud pattern does this match? Describe the pattern in plain language. Quote the specific SHAP features as evidence.	1–2 paragraphs
4. Network Context	Graph analysis: is this VPA connected to known fraud accounts? Is it in a suspicious cluster? What is the fund forwarding chain?	1 paragraph
5. Analyst Recommended Actions	Step-by-step: what should the fraud analyst do next? Call customer? Check KYC? Escalate to NPCI? Timeline for action?	3–5 bullet points

3.2.6 Payment Gateway Webhook Simulator

Build a separate FastAPI microservice that simulates the payment gateway's fraud hold/block API:

- ❑ Expose POST /hold — accepts { txn_id, risk_score, reason } and responds { status: 'HELD', hold_expires_at } — transaction is held for 30 minutes pending analyst review.
- ❑ Expose POST /block — accepts { txn_id, risk_score, reason } and responds { status: 'BLOCKED', reason_code } — transaction is permanently blocked.
- ❑ Expose POST /release — allows an analyst to release a HELD transaction after review.
- ❑ Expose GET /status/{txn_id} — returns current hold/block status.
- ❑ Simulate 2% API failure rate (timeout / 503) — the fraud detection service must handle this with retry logic and a dead-letter queue.
- ❑ Log all hold and block events with full transaction context for audit purposes.

4. RECOMMENDED TECH STACK

Layer	Technology / Tool	Purpose
Transaction Stream	Apache Kafka + custom UPI simulator (Python)	Real-time UPI transaction stream
Feature Store	Redis (velocity counters, device cache, graph cache)	Sub-millisecond feature retrieval for fast path
ML — Anomaly	scikit-learn IsolationForest	Unsupervised anomaly detection
ML — Autoencoder	PyTorch (Dense Autoencoder)	Reconstruction-error anomaly detection
ML — Classifier	XGBoost + imbalanced-learn (SMOTE)	Supervised fraud classifier
Explainability	SHAP (shap library for XGBoost + KernelSHAP for others)	Per-prediction feature attribution
Graph Analytics	NetworkX + python-louvain + Redis graph cache	Fraud network detection + PageRank
GenAI Explanation	OpenAI GPT-4o / Anthropic Claude / Ollama	Streaming fraud analyst reports
Gateway Simulator	FastAPI (separate microservice)	Mock payment gateway hold/block API
Backend API	FastAPI + Uvicorn + sse-starlette + Celery	Async API + SSE streaming + background enrichment
Database	PostgreSQL (transactions, VPA history, fraud labels)	Persistent transaction and audit store
Model Tracking	MLflow	Model versioning + precision/recall/F1 tracking
Frontend	React + Vite + TailwindCSS + Recharts + react-force-graph	Live dashboard + network graph visualisation
Containerisation	Docker + Docker Compose	Single-command full system startup
Observability	Prometheus + Grafana + Loki + Promtail	Metrics, dashboards, structured logs

5. DATA COLLECTION & MODEL TRAINING STRATEGY

5.1 Building the Training Dataset

You must build your own labelled UPI transaction dataset. The following strategy is required:

1. Run the UPI simulator for 30 days of simulated time (time-accelerated) with a 0.3% fraud injection rate. This produces approximately 500,000+ transactions with ground-truth fraud labels. This is your primary dataset.
2. Reference Datasets for Structural Inspiration (do NOT use as training data directly):
 - (IEEE-CIS Fraud Detection dataset (Kaggle) — study feature engineering approaches used by top solutions.
 - (PaySim dataset — a financial transaction simulator that you can use to calibrate your own simulator's fraud patterns.
 - (RBI Annual Report on Payment Fraud (publicly available) — use statistics on Indian UPI fraud types to calibrate your fraud injection scenarios.
3. Label Engineering: The simulator injects fraud with ground-truth labels. However, you must also implement a semi-supervised labelling pass: run the trained Isolation Forest on all transactions and review the top 1% most anomalous unlabelled transactions manually — reclassify any that are genuinely suspicious but not injected fraud.
4. Train/Validation/Test Split: 70% training / 15% validation / 15% test. The test set must be temporally held out — use the last 4.5 simulated days as the test set. Never use future data to predict past transactions.
5. Class Imbalance Handling:
 - (SMOTE (Synthetic Minority Oversampling Technique): generate synthetic fraud examples in feature space for XGBoost training.
 - (Cost-sensitive learning: set XGBoost `scale_pos_weight` = $(\text{num_legitimate} / \text{num_fraud})$.
 - (Threshold tuning: after training, tune the classification threshold on the validation set to maximise F2-score (recall-weighted). Report the final threshold used.

5.2 Required Model Performance Targets

Metric	Target	Justification
Recall (Fraud Catch Rate)	$\geq 85\%$	Missing 15%+ of fraud is operationally unacceptable. Recall is the primary metric.
Precision	$\geq 40\%$	False positives block legitimate transactions — precision must be reasonable. Adjust threshold to balance.
F2-Score	≥ 0.65	F2 weights recall more heavily than precision — appropriate for fraud detection where false negatives are costlier.
False Positive Rate (FPR)	$\leq 1.0\%$	No more than 1 in 100 legitimate transactions should be blocked. Above this, customer experience degrades severely.
Inference Latency (p99)	$< 200\text{ms}$	Hard SLA — end-to-end from Kafka receive to risk tier assigned. Measured under 500 txns/second load.

AUROC	≥ 0.92	Area under the ROC curve — ensemble should significantly outperform any single model.
-------	-------------	---

5.3 Fraud Pattern Database

Build a fraud pattern catalog with at least 10 named patterns. Each pattern has: a name, description, key indicator features, typical ensemble score range, and recommended analyst action. The GenAI engine uses this catalog when matching detected anomalies to known patterns.

Pattern Name	Description	Key Features	Default Action
VELOCITY_SPIKE	50+ transactions from one VPA in 10 minutes	txn_count_10min > 50	Block + Alert
LARGE_AMOUNT_ANOMALY	Amount > 5x the sender's historical average	amount_zscore > 5.0	High-Risk Hold
NEW_DEVICE_LARGE_TXN	New device + amount > ₹10,000	is_new_device AND amount > 10000	High-Risk Hold
MULE_ACCOUNT_STAR	Receiving from 20+ senders in 1 hour	unique_senders_1hr > 20	Block + NPCI Report
RAPID_FORWARDING_CHAIN	Funds pass through 4+ VPAs in 5 minutes	chain_length >= 4	Block all nodes
GEO_IMPOSSIBLE	Transaction speed > 900km/h between locations	is_geo_impossible = True	Block + Alert
NIGHT_LARGE_TRANSFER	Large amount between 1AM–4AM	is_night AND amount > 50000	High-Risk Hold
VPA_IMPERSONATION	VPA name similar (edit distance < 3) to known brand	vpa_similarity_score > 0.85	Suspicious Alert
KNOWN_FRAUD_PROXIMITY	Receiver is 1 hop from a confirmed fraud VPA	fraud_hop_count = 1	Block
DORMANT_ACCOUNT_SPIKE	Account inactive 90+ days, suddenly active	days_since_last_txn > 90	Suspicious Alert

6. USER INTERFACE REQUIREMENTS

6.1 Application Screens

Screen 1 — Live Fraud Operations Centre (Core Screen)

This is the primary screen for a fraud analyst. Everything updates in real time via SSE:

- ❑ KPI Banner (top): Transactions per second (live counter), fraud alerts in last 60 minutes, transactions currently held at gateway, false positive rate (last 100 analyst-reviewed decisions).
- ❑ Live Transaction Heatmap: A scrolling table of the last 200 transactions, colour-coded by risk tier — green (Legitimate), amber (Suspicious), orange (High-Risk), red (Block). New transactions animate in at the top. Each row shows: transaction ID, sender/receiver VPA (masked), amount, risk score, and risk tier badge.
- ❑ Alert Queue Panel: All Suspicious / High-Risk / Block transactions that require analyst attention. Each alert card shows: transaction details, ensemble risk score, top 3 SHAP features, matched fraud pattern badge, and time since alert was raised. Action buttons: Confirm Fraud / Mark Legitimate / Escalate.
- ❑ GenAI Explanation Panel: On selecting any alert, the 5-section GenAI fraud report streams token-by-token in the right panel. Re-selecting a different alert re-triggers a new stream.
- ❑ Model Performance Widgets: Three mini-gauges showing each model's current individual fraud catch rate (updated on each analyst decision). Drift indicator: red if any model's catch rate has dropped > 10% from baseline.

Screen 2 — Fraud Network Graph

- ❑ An interactive force-directed graph (using react-force-graph or D3.js) showing the transaction network for the last 30 minutes.
- ❑ Nodes: VPAs — colour-coded by risk (green/amber/orange/red). Size proportional to PageRank (larger = more central = higher risk).
- ❑ Edges: transactions — colour-coded by amount (thin = small, thick = large). Direction shown with arrows.
- ❑ Hover on a node: shows VPA ID, total transactions, total amount sent/received, community ID, risk score.
- ❑ Click on a node: highlights all transactions involving this VPA and shows the GenAI explanation for the highest-risk transaction.
- ❑ Detected fraud clusters are highlighted with a red background ring — community detection results rendered visually.
- ❑ Rapid forwarding chains highlighted in pulsing orange — animate the money flow path.

Screen 3 — Model Performance & Explainability

- ❑ Confusion Matrix: live-updated as analysts confirm/reject fraud alerts. Shows TP, TN, FP, FN counts and rates.
- ❑ Precision-Recall Curve: plotted for each individual model and the ensemble — with the current operating threshold marked.
- ❑ SHAP Summary Plot: Global feature importance across the last 1,000 scored transactions — which features are driving the most fraud detections today?
- ❑ SHAP Waterfall Chart: For the selected transaction — shows exactly how each feature pushed the XGBoost score up or down from the base value.
- ❑ Model Drift Monitor: Rolling 1-hour fraud catch rate vs. the 7-day baseline. Red alert if drift exceeds threshold.
- ❑ Threshold Tuner: An interactive slider allowing the analyst to adjust the risk tier thresholds and see the impact on precision, recall, and estimated blocked legitimate transactions per hour.

Screen 4 — Transaction Investigator

- ❑ Search any transaction by ID, VPA, device ID, or amount range.
- ❑ Full transaction detail view: all raw fields + all computed features + each model's score + SHAP waterfall.
- ❑ Transaction timeline: all transactions for a given VPA in chronological order, with risk scores visualised.
- ❑ Network neighbourhood: show the 2-hop transaction graph around a selected VPA.
- ❑ Analyst notes: free-text field for documenting investigation findings, linked to the transaction record.

Screen 5 — Observability Dashboard

- ❑ Embedded Grafana panels: inference latency (p50/p95/p99), Kafka consumer lag, false positive rate trend, fraud alert volume per hour, gateway hold/block counts.
- ❑ MLflow model metrics: current production model version, training data size, AUROC, F2-score, precision, recall.

6.2 UX Requirements

- ❑ Inference latency indicator: every transaction card in the heatmap shows the time from Kafka receive to risk tier assigned — must show green (< 200ms), amber (200–500ms), red (> 500ms).
- ❑ Alert cards must appear in the Alert Queue within 1 second of a Suspicious/High-Risk/Block classification.
- ❑ GenAI explanation must begin streaming within 3 seconds of an alert being selected.
- ❑ Network graph must update within 5 seconds of new transactions arriving — incremental update, not full re-render.
- ❑ The Threshold Tuner slider must recalculate precision/recall in real time (< 300ms) using pre-computed score distributions.
- ❑ All analyst decisions (Confirm Fraud / Mark Legitimate) must be logged with timestamp and used to update confusion matrix metrics immediately.

7. OBSERVABILITY & METRICS

7.1 Required Metrics

Metric	Description	Tool	Alert Threshold
inference_latency_ms	End-to-end: Kafka receive → risk tier assigned (p50, p95, p99)	Prometheus Histogram	p99 > 200ms = alert
fraud_alert_rate	Alerts per minute by risk tier (Suspicious / High-Risk / Block)	Prometheus Gauge	> 3x baseline = drift alert
false_positive_rate	% of analyst-confirmed 'Mark Legitimate' among all alerts reviewed	Prometheus Gauge	> 5% = alert

fraud_catch_rate	% of confirmed fraud caught by the model (rolling 1-hour window)	Prometheus Gauge	< 80% = critical alert
ensemble_score_distribution	Histogram of ensemble scores — monitors score distribution drift	Prometheus Histogram	Mean shift > 0.05 = alert
gateway_webhook_latency_ms	Round-trip time to POST hold/block request to Gateway Mock API	Prometheus Histogram	> 500ms = alert
gateway_webhook_failure_rate	% of gateway API calls that fail (timeout / 5xx)	Prometheus Gauge	> 3% = alert
kafka_consumer_lag	Lag on the upi_transactions consumer group	Prometheus Gauge	> 5000 = alert
genai_rationale_first_token_ms	Latency from alert triggered to first token in analyst dashboard	Prometheus Gauge	> 3s = alert
model_feature_drift	PSI (Population Stability Index) for top-10 features vs. training dist.	Prometheus Gauge	PSI > 0.2 = alert

7.2 Grafana Dashboards

- Dashboard 1 — Real-Time Fraud Operations: Transactions per second (live), fraud alert rate by tier, current false positive rate, Kafka consumer lag, gateway hold/block counts, inference latency p99.
- Dashboard 2 — Model Performance: Fraud catch rate trend (rolling 1hr), precision-recall over time, AUROC, confusion matrix counts, model drift indicator (PSI per feature), threshold sensitivity chart.
- Dashboard 3 — System Health & Pipeline: Kafka consumer lag trend, gateway webhook failure rate, GenAI rationale latency, active SSE connections, PostgreSQL write throughput, Redis memory usage.

7.3 MLflow Model Tracking

- Every training run logged: model type, SMOTE config, class weight, hyperparameters, train/val/test AUROC, F1, F2, precision, recall, FPR.
- All three model types (IsolationForest, Autoencoder, XGBoost) registered in the MLflow Model Registry with ensemble weights documented.
- Ensemble weights tuned on validation set logged as MLflow parameters — show the grid search or Bayesian optimisation that found the optimal weights.
- When analyst decisions accumulate (every 500 corrections), a model retraining run is logged comparing the retrained model against the current production model before promotion.

8. 2-WEEK PROJECT TIMELINE

Week	Days	Milestone	Deliverable
Week 1	Day 1	Environment + Simulator Setup	Docker Compose up; Kafka running; UPI simulator publishing 500 txns/sec to Kafka
Week 1	Day 1–2	Feature Engineering Pipeline	All 8 feature groups computing; Redis velocity counters working; features stored in DB
Week 1	Day 2–3	Isolation Forest Training	Model trained on legitimate txns; anomaly scores calibrated; MLflow logged
Week 1	Day 3–4	Autoencoder Training (PyTorch)	Autoencoder trained; reconstruction error normalised; per-txn scores computed
Week 1	Day 4–5	XGBoost Classifier + SMOTE + SHAP	XGBoost trained with SMOTE; F2-score ≥ 0.65 ; SHAP values computed per prediction
Week 1	Day 5–6	Ensemble Combiner + Risk Tier Assignment	Three-model ensemble live; weighted scores; four-tier risk classification working
Week 1	Day 6–7	Graph Analyser + Gateway Webhook Sim	NetworkX graph; Louvain clustering; star/chain detection; gateway hold/block API live
Week 2	Day 8	GenAI Fraud Explanation Streaming	5-section GenAI report streaming via SSE for every Suspicious+ transaction
Week 2	Day 9	React UI — Fraud Operations Centre	Live heatmap, alert queue, GenAI panel, KPI banner all rendering and updating live
Week 2	Day 10	React UI — Network Graph Screen	react-force-graph rendering transaction network; clusters highlighted; node drill-down
Week 2	Day 11	React UI — Model Performance + Investigator	Confusion matrix, SHAP waterfall, threshold slider, transaction search all working
Week 2	Day 12	Observability Stack + Drift Detection	All 10 metrics live; 3 Grafana dashboards; PSI drift detection; MLflow all 3 models

Week 2	Day 13	Load Testing + Demo Prep	System validated at 500 txns/sec; p99 < 200ms confirmed; demo scenarios rehearsed
Week 2	Day 14	Final Demo Day	Live demo: streaming transactions + fraud injection + alerts + GenAI + network graph + Grafana

9. DEMO REQUIREMENTS & EVALUATION CRITERIA

9.1 Demo Structure (Day 14 — 50 Minutes)

Part 1 — Architecture Walkthrough (10 minutes)

Walk through your full architecture. Be prepared to answer in depth:

- ☐ Why use three different model types instead of one? What does each model catch that the others miss?
- ☐ How do you handle the < 200ms SLA? Which features are in the fast path vs. the async enrichment path?
- ☐ Explain the SMOTE approach — why is it needed, and what are its limitations?
- ☐ How does the graph analyser escalate risk scores asynchronously? What prevents a race condition between the fast-path score and the graph escalation?
- ☐ Walk through the ensemble weight selection — why 25/25/50? Show the grid search results from MLflow.
- ☐ What is Population Stability Index (PSI) and how does your drift detection use it?

Part 2 — Live Demo (25 minutes)

All demos use a live running system at 500 transactions per second.

6. Show the Kafka consumer group in a terminal — transactions streaming in at high rate.
7. Show the Live Fraud Operations Centre heatmap — transactions colour-coded, new ones animating in at the top.
8. Trigger a VELOCITY_SPIKE fraud scenario — inject 60 transactions from one VPA in 2 minutes. Watch the alert appear in the queue.
9. Select the alert — show the GenAI 5-section explanation streaming token-by-token. Point out the SHAP features cited.
10. Trigger a MULE_ACCOUNT_STAR scenario — watch the gateway webhook fire (Block request). Show the gateway simulator log receiving the block request.
11. Navigate to the Network Graph screen — show the mule account node highlighted in red, surrounded by its phishing victim senders.
12. Trigger a RAPID_FORWARDING_CHAIN — show the animated chain highlighted in orange in the network graph.
13. Switch to the Model Performance screen — show the confusion matrix, SHAP summary plot, and threshold slider. Move the slider and show precision/recall reacting live.
14. Show the Transaction Investigator — search for a flagged VPA and show its full transaction timeline and 2-hop network neighbourhood.
15. Show the Grafana dashboards — inference latency p99, fraud alert rate, consumer lag.

16. Simulate a gateway API failure — show the dead-letter queue handling and retry logic.

Part 3 — Methodology Deep Dive (10 minutes)

- ☐ Show the MLflow experiment: three model runs, ensemble weight tuning, final F2-score comparison.
- ☐ Open the Precision-Recall curve for the ensemble vs. individual models — show quantitatively that the ensemble outperforms each individual model.
- ☐ Walk through the SHAP waterfall chart for one specific fraud transaction — explain each feature's contribution in plain language.
- ☐ Show the Louvain community detection result — open a visualisation of one detected fraud cluster from your test data.
- ☐ Show the SMOTE class distribution before vs. after — how did oversampling change the training data balance?
- ☐ Walk through the threshold tuning process — what F2-score did you achieve at different thresholds, and why did you choose the final threshold?

Part 4 — Q&A (5 minutes)

Open questions from the assessor on any aspect of the system.

9.2 Scoring Rubric

Criterion	What We Look For	Max Score
Multi-Model Ensemble	All 3 models trained, SHAP working, ensemble outperforms individual models, AUROC ≥ 0.92 , F2 ≥ 0.65	25
Real-Time Inference Pipeline	< 200ms p99 at 500 txns/sec, feature fast path vs. async enrichment, risk tier assignment, gateway webhook	25
Graph Fraud Network	NetworkX graph, Louvain clustering, star/chain detection, UI network graph with cluster highlighting	15
GenAI Fraud Explanation	All 5 sections present, SHAP values cited, pattern matched, streaming visible, analyst actions recommended	15
UI — Fraud Ops Centre	Live heatmap, alert queue, SHAP waterfall, threshold slider, network graph interactive, all states handled	10
Observability & Drift	10 metrics, 3 dashboards, PSI drift detection, MLflow for all 3 models, analyst decisions logged	10

TOTAL Minimum passing score: 70 / 100	100
--	------------

10. DELIVERABLES CHECKLIST

10.1 Code Repository Structure

```
upi-fraud-detection/
├── simulator/                                # UPI transaction generator
│   ├── producer.py                         # Kafka producer - 500 txns/sec
│   ├── fraud_injector.py                   # 10 fraud pattern injection scenarios
│   ├── vpa_generator.py                    # 10,000 realistic Indian UPI VPAs
│   └── Dockerfile
├── feature-service/                         # Feature engineering pipeline
│   ├── app.py                             # Kafka consumer + feature computation
│   ├── velocity.py                        # Redis sliding window velocity features
│   ├── device.py                          # Device consistency + risk features
│   ├── graph_features.py                   # NetworkX PageRank + community features
│   ├── geo.py                             # Geo-impossible detection
│   └── Dockerfile
├── ml-service/                             # Multi-model ensemble inference
│   ├── app.py                             # FastAPI /score endpoint + Kafka consumer
│   ├── models/
│   │   ├── isolation_forest.py            # Isolation Forest training + inference
│   │   ├── autoencoder.py                 # PyTorch Autoencoder
│   │   └── xgboost_model.py               # XGBoost + SMOTE + SHAP
│   ├── ensemble.py                        # Weighted ensemble combiner
│   ├── risk_tier.py                       # Four-tier classification + thresholds
│   ├── drift_monitor.py                   # PSI computation per feature
│   ├── mlflow_client.py                   # Experiment logging
│   └── Dockerfile
├── graph-service/                          # Fraud network analyser (async)
│   ├── app.py                             # Celery task + NetworkX graph manager
│   ├── community.py                       # Louvain + star + chain detection
│   ├── escalator.py                       # Score escalation publisher
│   └── Dockerfile
├── genai-service/                          # Fraud explanation engine
│   ├── explainer.py                       # Context builder + LLM streaming client
│   ├── pattern_matcher.py                 # Fraud pattern DB matching logic
│   ├── prompts.py                         # 5-section analyst report prompt
│   └── Dockerfile
├── gateway-simulator/                      # Payment gateway mock API
│   ├── app.py                             # FastAPI: /hold /block /release /status
│   ├── failure_sim.py                     # 2% failure injection + jitter
│   └── Dockerfile
├── api-gateway/                            # Main FastAPI app + SSE
│   ├── main.py                            # Routes + SSE orchestration
│   ├── metrics.py                         # Prometheus metrics definitions
│   └── Dockerfile
├── frontend/                              # React + Vite + TailwindCSS
│   ├── src/components/                    # FraudHeatmap, AlertQueue, NetworkGraph ...
│   └── Dockerfile
├── observability/                          # Prometheus + Grafana + Loki
├── notebooks/                             # Model training + SHAP analysis + EDA
├── tests/                                  # pytest + Playwright E2E + load tests
├── docker-compose.yml
├── .env.example
└── README.md
```

10.2 Documentation

- ❑ README.md: Full setup guide, how to run the simulator, how to trigger each fraud scenario, how to run load tests, environment variables.
- ❑ Architecture diagram committed as PNG.
- ❑ Model Card (all three models): Training data stats, class distribution, SMOTE config, hyperparameters, AUROC / F2 / precision / recall on test set, ensemble weights, known failure modes.
- ❑ Fraud Pattern Catalog: All 10 patterns documented — description, feature indicators, risk tier mapping, recommended analyst action.
- ❑ Load Test Report: Inference latency percentiles (p50/p95/p99) measured at 100 / 300 / 500 txns/sec. Screenshot of Grafana latency dashboard under load.

10.3 Demo Materials

- ❑ Slide deck (max 8 slides) for the architecture walkthrough.
- ❑ Fraud scenario script: a prepared sequence of 11 demo steps (Section 9.1) with the specific fraud patterns to inject, the timing, and the expected system response for each.

11. TIPS, HINTS & COMMON PITFALLS

11.1 Tips for Success

- ❑ **Profile your feature engineering pipeline on Day 2. If Redis velocity counter lookups take > 20ms, your entire fast path budget is gone before inference even starts. Use Redis pipelines (batch commands) to retrieve all velocity features in a single round trip.**
- ❑ **Train the Isolation Forest and Autoencoder ONLY on legitimate transactions. Contaminating training data with fraud samples defeats the purpose of unsupervised anomaly detection.**
- ❑ **Use ONNX to export both the Isolation Forest and XGBoost models. ONNX runtime inference is significantly faster than scikit-learn / XGBoost native inference and will help you hit the 200ms SLA.**
- ❑ **Build the fraud scenario injector as a configurable module with named scenarios (VELOCITY_SPIKE, MULE_ACCOUNT_STAR, etc.) that can be triggered via the UI during the demo. This makes the demo far more impressive than manually crafting test transactions.**
- ❑ **The SHAP waterfall chart is the most impressive explainability visualisation you can show. Implement it using the shap library's native matplotlib output, then export to PNG and serve it as a base64 image to the frontend — or use SHAP.js for a native React implementation.**
- ❑ **Test the gateway webhook retry logic early. A 2% gateway failure rate means roughly 10 failed webhook calls per second at peak load. If you haven't built the dead-letter queue, you'll lose those hold/block requests silently.**

11.2 Common Pitfalls to Avoid

- ❑ Reporting 99.9% accuracy as your headline metric — this is trivially achieved by classifying everything as legitimate. Always report F2-score, recall, and false positive rate. The assessor will ask specifically about these.
- ❑ Training XGBoost on the full imbalanced dataset without SMOTE or class weights — the model will learn to always predict legitimate and will miss nearly all fraud.
- ❑ Building all feature computation synchronously in the fast path — graph features, geo-impossible checks, and VPA age lookups must be async. Only velocity, amount z-score, and temporal features belong in the 200ms fast path.
- ❑ Using a single global graph that is fully rebuilt on every transaction — this will not scale. Maintain an incremental graph with time-windowed edges (only keep edges from the last 30 minutes).
- ❑ Not masking VPA IDs in the analyst UI — never show complete VPA addresses in demo screenshots or presentation slides. Mask to user***@bank format.
- ❑ Forgetting to validate that the three models are trained on the same feature schema — if the feature pipeline changes after model training, inference will silently fail or produce garbage scores.

11.3 Suggested Learning Resources

- ❑ SHAP documentation and tutorials: shap.readthedocs.io
- ❑ PyTorch Autoencoder for anomaly detection: pytorch.org/tutorials
- ❑ imbalanced-learn (SMOTE) documentation: imbalanced-learn.org
- ❑ python-louvain (community detection): github.com/taynaud/python-louvain
- ❑ react-force-graph for network visualisation: github.com/vasturiano/react-force-graph
- ❑ ONNX runtime Python: onnxruntime.ai/docs
- ❑ Population Stability Index explainer: towardsdatascience.com/psi-and-csi-top-2-model-monitoring-metrics

A NOTE FROM YOUR ASSESSOR

Every millisecond matters. Every missed fraud transaction costs someone their savings.

The hardest problem in this project is not building a model that detects fraud —
it is building one that does it fast enough to matter, at scale,
with low enough false positives that real users are not blocked from their own money,
and with explanations clear enough that a fraud analyst can act on them in seconds.

That is what production fraud detection looks like. Build it.

Protect every rupee. Ship it.